Advanced Topics in Probabilistic Machine Learning: Concepts, Derivations, and Applications

Introduction

This report aims to provide a comprehensive exploration of several advanced topics within probabilistic machine learning. Its primary objective is to deliver detailed solutions to a series of challenging problems while simultaneously offering thorough conceptual explanations of the underlying principles. By dissecting complex theories and mathematical derivations, this document endeavors to foster a deep and intuitive understanding of concepts such as Maximum Likelihood Estimation (MLE), Kullback-Leibler (KL) Divergence, the relationship between generative and discriminative models like Naive Bayes and Logistic Regression, the role of conditional independence in parameterizing distributions, the intricacies of autoregressive models, the application of Monte Carlo methods for integration, and the practical use of Long Short-Term Memory (LSTM) networks for text generation. The report is structured problem by problem, with each section dedicated to a specific challenge and the foundational knowledge required to address it effectively.

---

1 Problem 1: Maximum Likelihood Estimation and KL Divergence

This problem explores the fundamental connection between maximizing the likelihood of observed data and minimizing the Kullback-Leibler (KL) divergence between the empirical data distribution and a model distribution. Understanding this equivalence is crucial for appreciating why MLE is a cornerstone of statistical inference and machine learning.

1.1 A. Conceptual Deep Dive: Foundations of Parameter Estimation and Model Comparison

1.1.1 Maximum Likelihood Estimation (MLE) Explained

Maximum Likelihood Estimation (MLE) is a central method in statistics and machine learning for estimating the parameters of an assumed probability distribution, given some observed data. The core principle is to select parameter values that make the observed data most probable under the assumed statistical model.[1]

The **likelihood function**, denoted as $L(\theta|D)$, quantifies the probability (or probability density for continuous data) of observing the data D given a specific set of parameters θ. It is important to note that the likelihood is a function of the parameters θ, with the data D being fixed.[1] If the data consists of N independent and identically distributed (i.i.d.) observations $x_1, x_2, \ldots, x_N$, the likelihood function is the product of the probabilities of each observation:

$$L(\theta|D) = \prod_{i=1}^{N} p(x_i|\theta)$$

In practice, it is often more convenient to work with the **log-likelihood function**, $\mathcal{L}(\theta|D) = \log L(\theta|D)$. Since the logarithm is a monotonically increasing function, maximizing the log-likelihood is equivalent to maximizing the likelihood itself.[1] The log-likelihood transforms the product of probabilities into a sum:

$$\mathcal{L}(\theta|D) = \sum_{i=1}^{N} \log p(x_i|\theta)$$

This transformation offers several advantages, including simpler analytical derivations (derivatives of sums are easier than products) and improved numerical stability, as multiplying many small probabilities can lead to underflow.

The goal of MLE is to find the parameter values $\hat{\theta}_{\text{MLE}}$ that maximize this (log-)likelihood function over the permissible parameter space Θ:[2]

$$\hat{\theta}_{\text{MLE}} = \arg\max_{\theta \in \Theta} L(\theta|D) = \arg\max_{\theta \in \Theta} \mathcal{L}(\theta|D)$$

MLE is a specific instance of an extremum estimator, where the objective function being maximized is the likelihood.[1] While MLE possesses desirable asymptotic properties, such as consistency (the estimator converges to the true parameter value as the sample size increases) and asymptotic normality,[1] its performance in finite samples can sometimes be surpassed by other estimators.

A key consideration when using MLE is that it operates under the assumption of a chosen parametric family for the model p_θ. MLE identifies the parameters that best explain the observed data *within that family*. However, in practical applications, the data is rarely, if ever, generated perfectly by the chosen model p_θ. Instead, p_θ serves as an often idealized representation of the true data-generating process.[1] This implies that the "optimality" of MLE parameters is relative to the chosen model class. If the model class itself is a poor fit for the underlying reality, even the best parameters found by MLE might yield a model that does not accurately reflect or generalize well from the data. This underscores the distinct but related importance of model selection in the overall modeling pipeline.

1.1.2 Kullback-Leibler (KL) Divergence Explained

The Kullback-Leibler (KL) divergence, also known as relative entropy, is a measure of how one probability distribution, P, diverges from a second, reference probability distribution, Q. For discrete probability distributions $p(x)$ and $q(x)$ defined over the same probability space $\mathcal{X}$, the KL divergence of Q from P (often denoted $D_{\text{KL}}(P \parallel Q)$ or $D_{\text{KL}}(p(x) \parallel q(x))$) is defined as:

$$D_{\text{KL}}(p(x) \parallel q(x)) = \sum_{x \in \mathcal{X}} p(x) \log \frac{p(x)}{q(x)} = \mathbb{E}_{x \sim p(x)}[\log p(x) - \log q(x)]$$

For continuous distributions, the sum is replaced by an integral.

A fundamental interpretation of KL divergence is as a measure of **information loss** when $q(x)$ is used to approximate $p(x)$.[4] If $p(x)$ is the true distribution of data, and $q(x)$ is a model used to approximate $p(x)$, then $D_{\text{KL}}(p(x) \parallel q(x))$ represents the expected number of extra bits required to encode samples from $p(x)$ using a code optimized for $q(x)$ rather than one optimized for $p(x)$.

KL divergence has several important properties:

1. **Non-negativity**: $D_{\text{KL}}(p(x) \parallel q(x)) \geq 0$. This is a consequence of Gibbs' inequality.[5]
2. **Identity of Indiscernibles**: $D_{\text{KL}}(p(x) \parallel q(x)) = 0$ if and only if $p(x) = q(x)$ almost everywhere. This means the divergence is zero only when the two distributions are identical.[5]

3. **Asymmetry**: In general, $D_{\mathrm{KL}}\left(p(x) \,\|\, q(x)\right) \neq D_{\mathrm{KL}}\left(q(x) \,\|\, p(x)\right)$. This asymmetry means that KL divergence is not a true distance metric (which must be symmetric). The choice of which distribution is P and which is Q matters.[5]

The KL divergence can be related to cross-entropy. The cross-entropy between $p(x)$ and $q(x)$ is $H(p, q) = \mathbb{E}_{x\sim p(x)}[-\log q(x)]$. The entropy of $p(x)$ is $H(p) = \mathbb{E}_{x\sim p(x)}[-\log p(x)]$. The KL divergence can then be written as:[5]

$$D_{\mathrm{KL}}\left(p(x) \,\|\, q(x)\right) = H(p, q) - H(p)$$

This shows that minimizing $D_{\mathrm{KL}}\left(p(x) \,\|\, q(x)\right)$ with respect to $q(x)$ (when $p(x)$ is fixed) is equivalent to minimizing the cross-entropy $H(p, q)$, because $H(p)$ does not depend on $q(x)$.

1.1.3 The Equivalence of Maximizing Likelihood and Minimizing KL Divergence

A profound result in statistical modeling is the equivalence between maximizing the likelihood of data under a model $p_\theta(y|x)$ and minimizing the KL divergence between the empirical data distribution $\hat{p}(y|x)$ and the model distribution $p_\theta(y|x)$.[5]

To see this, let $\hat{p}(x, y)$ be the empirical distribution of the data, and $\hat{p}(y|x) = \hat{p}(x, y)/\hat{p}(x)$ be the empirical conditional distribution. We want to find parameters θ for our model $p_\theta(y|x)$ that best fit this empirical distribution. Consider the KL divergence $D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)$ averaged over the empirical distribution of x, $\hat{p}(x)$:

$$\begin{aligned}
\mathbb{E}_{\hat{p}(x)}\left[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)\right] &= \mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log \frac{\hat{p}(y|x)}{p_\theta(y|x)}\right] \\
\text{Expanding the logarithm:} &= \mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log \hat{p}(y|x) - \sum_y \hat{p}(y|x) \log p_\theta(y|x)\right] \\
&= \underbrace{\mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log \hat{p}(y|x)\right]}_{\text{Term 1: Does not depend on } \theta} - \underbrace{\mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log p_\theta(y|x)\right]}_{\text{Term 2: Depends on } \theta}
\end{aligned}$$

The first term is the negative conditional entropy of y given x under the empirical distribution, $-\mathbb{E}_{\hat{p}(x)}[H(\hat{p}(Y|X = x))]$. Crucially, this term does not depend on the model parameters θ.[6] Therefore, minimizing the entire expression with respect to θ is equivalent to minimizing the negative of the second term, or equivalently, maximizing the second term:

$$\arg\min_\theta \mathbb{E}_{\hat{p}(x)}\left[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)\right] = \arg\min_\theta \left(-\mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log p_\theta(y|x)\right]\right) = \arg\max_\theta \mathbb{E}_{\hat{p}(x)}\left[\sum_y \hat{p}(y|x) \log\right.$$

The expression $\mathbb{E}_{\hat{p}(x)}[\sum_y \hat{p}(y|x) \log p_\theta(y|x)]$ can be rewritten as an expectation over the joint empirical distribution $\hat{p}(x, y)$:

$$\sum_x \hat{p}(x) \sum_y \hat{p}(y|x) \log p_\theta(y|x) = \sum_x \sum_y \hat{p}(x)\hat{p}(y|x) \log p_\theta(y|x) = \sum_x \sum_y \hat{p}(x, y) \log p_\theta(y|x) = \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]$$

For a finite dataset $\{(x_i, y_i)\}_{i=1}^N$, the empirical distribution $\hat{p}(x, y)$ assigns probability $1/N$ to each observed pair (if all are unique, or count/N more generally). Thus, the expectation becomes an average:

$$\mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)] = \frac{1}{N} \sum_{i=1}^N \log p_\theta(y_i|x_i)$$

Maximizing this sum is precisely the objective of Maximum Likelihood Estimation.

1.2 B. Solution and Derivation for Problem 1

Objective: Show that the following equivalence holds:

$$\arg\max_{\theta\in\Theta} \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)] = \arg\min_{\theta\in\Theta} \mathbb{E}_{\hat{p}(x)}[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)]$$

Step 1: Expand the Right-Hand Side (RHS)

Let us start with the expression on the RHS that we want to minimize:

$$\mathbb{E}_{\hat{p}(x)}[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)]$$

By definition of KL-divergence, $D_{\mathrm{KL}}\left(p(z) \,\|\, q(z)\right) = \mathbb{E}_{z\sim p(z)}[\log p(z) - \log q(z)]$. Applying this to our conditional distributions, for a given x:

$$D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right) = \sum_y \hat{p}(y|x)\left(\log \hat{p}(y|x) - \log p_\theta(y|x)\right)$$

(Assuming Y is discrete. For continuous Y, the sum becomes an integral, but the argument remains the same.) Now, take the expectation with respect to $\hat{p}(x)$:

$$\mathbb{E}_{\hat{p}(x)}[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)] = \sum_x \hat{p}(x)\left[\sum_y \hat{p}(y|x)\left(\log \hat{p}(y|x) - \log p_\theta(y|x)\right)\right]$$

Step 2: Decompose the Log Term and Distribute Sums

Distribute $\hat{p}(y|x)$ inside the parenthesis and then distribute $\hat{p}(x)$ across the terms:

$$\begin{aligned}\mathbb{E}_{\hat{p}(x)}[D_{\mathrm{KL}}\left(\hat{p}(y|x) \,\|\, p_\theta(y|x)\right)] &= \sum_x \hat{p}(x)\left[\sum_y \hat{p}(y|x)\log \hat{p}(y|x) - \sum_y \hat{p}(y|x)\log p_\theta(y|x)\right] \\ &= \sum_x \hat{p}(x)\sum_y \hat{p}(y|x)\log \hat{p}(y|x) - \sum_x \hat{p}(x)\sum_y \hat{p}(y|x)\log p_\theta(y|x)\end{aligned}$$

Step 3: Separate Terms and Identify Constants with Respect to θ

The first term is:

$$T_1 = \sum_x \sum_y \hat{p}(x)\hat{p}(y|x)\log \hat{p}(y|x) = \sum_x \sum_y \hat{p}(x,y)\log \hat{p}(y|x)$$

This term is the negative conditional entropy of Y given X under the empirical distribution $\hat{p}$. Importantly, T_1 does not depend on the model parameters θ. It is solely a property of the empirical data distribution.

The second term is:

$$T_2 = \sum_x \sum_y \hat{p}(x)\hat{p}(y|x)\log p_\theta(y|x) = \sum_x \sum_y \hat{p}(x,y)\log p_\theta(y|x) = \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]$$

So, the expression we are minimizing is $T_1 - T_2$.

Step 4: Analyze the Optimization

We want to find:

$$\arg\min_{\theta\in\Theta}(T_1 - T_2) = \arg\min_{\theta\in\Theta}\left(\sum_x \sum_y \hat{p}(x,y)\log \hat{p}(y|x) - \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]\right)$$

Since T_1 does not depend on θ, minimizing $T_1 - T_2$ with respect to θ is equivalent to minimizing $-T_2$ with respect to θ:

$$\arg\min_{\theta\in\Theta}(-T_2) = \arg\min_{\theta\in\Theta}\left(-\mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]\right)$$

Minimizing the negative of a quantity is equivalent to maximizing the quantity itself:

$$= \arg\max_{\theta\in\Theta} \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]$$

Step 5: Relate to the Left-Hand Side (LHS)
The expression derived in Step 4, $\arg\max_{\theta\in\Theta} \mathbb{E}_{\hat{p}(x,y)}[\log p_\theta(y|x)]$, is exactly the left-hand side (LHS) of the equivalence we aimed to prove.

Conclusion: The derivation shows that maximizing the expected log-likelihood under the empirical data distribution is equivalent to minimizing the expected KL divergence between the empirical conditional distribution and the model's conditional distribution.

2 Problem 2: Logistic Regression and Naive Bayes

This problem delves into the relationship between two popular classification algorithms: Gaussian Naive Bayes (a generative model) and Logistic Regression (a discriminative model). It specifically asks to demonstrate that under certain conditions, a Gaussian Naive Bayes model can be represented by an equivalent multi-class Logistic Regression model.

2.1 A. Conceptual Deep Dive: Generative vs. Discriminative Classifiers

2.1.1 Generative vs. Discriminative Models

Machine learning models for classification can be broadly categorized into generative and discriminative approaches, based on how they learn from the data.[8]

Generative models aim to learn the joint probability distribution $p(X, Y)$ of the inputs X and labels Y. Often, they achieve this by modeling the class-conditional probability $p(X|Y)$ and the prior probability of classes $p(Y)$ separately. Once these are learned, the posterior probability $p(Y|X)$ can be computed using Bayes' theorem: $p(Y|X) = \frac{p(X|Y)p(Y)}{p(X)}$, where $p(X) = \sum_Y p(X|Y)p(Y)$ is the marginal probability of X (evidence).[8] Because they model how the data for each class is generated, generative models can, in principle, be used to synthesize new data points resembling the training data.[10] Examples include Naive Bayes, Gaussian Mixture Models, and Hidden Markov Models.

Discriminative models, on the other hand, directly learn the conditional probability $p(Y|X)$ or learn a direct mapping from inputs X to labels Y, effectively learning the decision boundary between classes.[10] They do not attempt to model the underlying distribution of the inputs X. Examples include Logistic Regression, Support Vector Machines, and traditional feed-forward Neural Networks.

2.1.2 Gaussian Naive Bayes (GNB) Explained

Naive Bayes classifiers are a family of simple probabilistic classifiers based on applying Bayes' theorem with a "naive" assumption of conditional independence between features.[13] Given a class label y and a feature vector $x = (x_1, x_2, \ldots, x_n)$, the Naive Bayes assumption states that the features are conditionally independent:

$$p(x_1, \ldots, x_n|y) = \prod_{i=1}^{n} p(x_i|y)$$

Gaussian Naive Bayes (GNB) is a specific type of Naive Bayes classifier used when features are continuous. It assumes that for each class y, the likelihood of each feature x_i, $p(x_i|y)$, follows a Gaussian (normal) distribution.[12] In the context of this problem, the model is specified as:

1. $p_\theta(y) = \pi_y$, where $\sum_{y=1}^{k} \pi_y = 1$. This is the prior probability of class y.
2. $p_\theta(x|y) = \mathcal{N}(x|\mu_y, \sigma^2 I)$. This is the class-conditional density for the n-dimensional feature vector x. It implies that given class y, x is drawn from a multivariate Gaussian distribution with mean vector μ_y and a covariance matrix $\sigma^2 I$. The covariance matrix $\sigma^2 I$ signifies that, given the class y, the features x_j are uncorrelated and share the same variance σ^2. Notably, the variance σ^2 is shared across all classes.

2.1.3 Multi-class Logistic Regression Explained

Logistic Regression is a discriminative classification algorithm that directly models the posterior probability $p(y|x)$.[12] For multi-class classification with k classes, the model typically uses the

softmax function to ensure that the probabilities for all classes sum to 1. The form given in the problem is:

$$p_\gamma(y|x) = \frac{\exp(x^T w_y + b_y)}{\sum_{j=1}^k \exp(x^T w_j + b_j)}$$

Here, $w_y \in \mathbb{R}^n$ is the weight vector and $b_y \in \mathbb{R}$ is the bias term associated with class y. The parameters γ of the model are $(w_1, \ldots, w_k, b_1, \ldots, b_k)$.

2.2 B. Solution and Derivation for Problem 2

Objective: Show that for any choice of GNB parameters $\theta = (\pi_1, \ldots, \pi_k, \mu_1, \ldots, \mu_k, \sigma)$, there exists a set of multi-class logistic regression parameters $\gamma = (w_1, \ldots, w_k, b_1, \ldots, b_k)$ such that $p_\theta(y|x) = p_\gamma(y|x)$.

Step 1: Write down $p_\theta(y|x)$ using Bayes' rule.

The posterior probability for class y given x in the GNB model is:

$$p_\theta(y|x) = \frac{p_\theta(x|y)p_\theta(y)}{\sum_{j=1}^k p_\theta(x|j)p_\theta(j)}$$

Substituting the given forms for $p_\theta(x|y) = \mathcal{N}(x|\mu_y, \sigma^2 I)$ and $p_\theta(y) = \pi_y$:

$$p_\theta(y|x) = \frac{\frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{1}{2\sigma^2}\|x - \mu_y\|^2\right) \pi_y}{\sum_{j=1}^k \frac{1}{(2\pi\sigma^2)^{n/2}} \exp\left(-\frac{1}{2\sigma^2}\|x - \mu_j\|^2\right) \pi_j}$$

The normalization constant $(2\pi\sigma^2)^{-n/2}$ appears in every term and cancels out:

$$p_\theta(y|x) = \frac{\exp\left(-\frac{1}{2\sigma^2}\|x - \mu_y\|^2\right) \pi_y}{\sum_{j=1}^k \exp\left(-\frac{1}{2\sigma^2}\|x - \mu_j\|^2\right) \pi_j}$$

Step 2: Expand the squared norm term and incorporate π_y.

The squared Euclidean norm is $\|x - \mu_y\|^2 = (x - \mu_y)^T(x - \mu_y) = x^T x - 2x^T\mu_y + \mu_y^T\mu_y$. The term in the numerator's exponent becomes:

$$-\frac{1}{2\sigma^2}(x^T x - 2x^T\mu_y + \mu_y^T\mu_y) + \log \pi_y$$

Let $A_y(x)$ be the argument of the exponent:

$$A_y(x) = -\frac{x^T x}{2\sigma^2} + \frac{x^T\mu_y}{\sigma^2} - \frac{\mu_y^T\mu_y}{2\sigma^2} + \log \pi_y$$

Then, $p_\theta(y|x) = \frac{\exp(A_y(x))}{\sum_{j=1}^k \exp(A_j(x))}$.

Step 3: Identify terms for logistic regression.

The term $-\frac{x^T x}{2\sigma^2}$ is common to all $A_y(x)$ for every class y. We can factor it out:

$$\exp(A_y(x)) = \exp\left(-\frac{x^T x}{2\sigma^2}\right) \exp\left(\frac{x^T\mu_y}{\sigma^2} - \frac{\mu_y^T\mu_y}{2\sigma^2} + \log \pi_y\right)$$

When we substitute this into the expression for $p_\theta(y|x)$, the common factor $\exp\left(-\frac{x^T x}{2\sigma^2}\right)$ cancels from the numerator and the denominator. Let $A'_y(x)$ be the remaining part of the exponent:

$$A'_y(x) = x^T\left(\frac{\mu_y}{\sigma^2}\right) + \left(-\frac{\mu_y^T\mu_y}{2\sigma^2} + \log \pi_y\right)$$

Then, $p_\theta(y|x) = \frac{\exp(A'_y(x))}{\sum_{j=1}^k \exp(A'_j(x))}$.

Step 4: Match with the multi-class logistic regression form.

The multi-class logistic regression model is given by $p_\gamma(y|x) = \frac{\exp(x^T w_y + b_y)}{\sum_{j=1}^{k} \exp(x^T w_j + b_j)}$. Comparing the form of $p_\theta(y|x)$ with $p_\gamma(y|x)$, we can establish an equivalence if we can define w_y and b_y in terms of the GNB parameters θ. We need $x^T w_y + b_y = A'_y(x)$.

$$x^T w_y + b_y = x^T \left(\frac{\mu_y}{\sigma^2}\right) + \left(-\frac{\mu_y^T \mu_y}{2\sigma^2} + \log \pi_y\right)$$

This equality holds for all x if we set:

$$w_y = \frac{\mu_y}{\sigma^2}$$
$$b_y = -\frac{\mu_y^T \mu_y}{2\sigma^2} + \log \pi_y = -\frac{\|\mu_y\|^2}{2\sigma^2} + \log \pi_y$$

Conclusion: For any given set of parameters $\theta = (\pi_1, \ldots, \pi_k, \mu_1, \ldots, \mu_k, \sigma)$ for the Gaussian Naive Bayes model (assuming $\sigma^2 > 0$ and $\pi_y > 0$), we can define a corresponding set of parameters $\gamma = (w_1, \ldots, w_k, b_1, \ldots, b_k)$ for the multi-class logistic regression model as shown above. With these definitions, $p_\theta(y|x) = p_\gamma(y|x)$ for all x and y.

3 Problem 3: Conditional Independence and Parameterization

This problem explores how the number of parameters needed to define a joint probability distribution over discrete random variables changes with different conditional independence assumptions.

3.1 A. Conceptual Deep Dive: Modeling Joint Distributions

3.1.1 Joint Probability Distributions and the Chain Rule

A **joint probability distribution** $P(X_1, \ldots, X_n)$ over a set of n random variables specifies the probability for every possible combination of their outcomes. If each variable X_i can take k_i distinct values, the total number of possible joint assignments is $\prod_{i=1}^{n} k_i$. Without any simplifying assumptions, defining this distribution requires specifying a probability for each of these joint states, minus one due to the constraint that all probabilities must sum to one.

The chain rule of probability provides a general way to decompose any joint distribution into a product of conditional probabilities:[15]

$$P(X_1, \ldots, X_n) = P(X_1) \prod_{i=2}^{n} P(X_i | X_1, \ldots, X_{i-1})$$

3.1.2 Conditional Independence and Its Role in Reducing Parameters

Conditional independence is a key concept for simplifying probabilistic models. Two variables A and B are said to be conditionally independent given a third variable C if knowing C renders A and B independent, i.e., $P(A|B, C) = P(A|C)$.[17] When conditional independence assumptions hold, they simplify the terms in the chain rule. For example, if X_i is conditionally independent of $X_1, \ldots, X_{i-2}$ given X_{i-1} (a Markov property), then $P(X_i|X_1, \ldots, X_{i-1})$ simplifies to $P(X_i|X_{i-1})$. Each such simplification reduces the number of parameters needed to specify its conditional distribution.[17]

3.2 B. Solution and Derivation for Problem 3

Let $X_1, \ldots, X_n$ be n discrete random variables, where X_i has k_i possible outcomes.

Part 1: No Conditional Independence Assumptions

Question: Without any conditional independence assumptions, what is the total number of independent parameters needed to describe the joint distribution over $(X_1, \ldots, X_n)$?

Solution: To specify the joint distribution $P(X_1, \ldots, X_n)$, we need to assign a probability to every possible combination of outcomes. The total number of distinct joint states is $\prod_{i=1}^{n} k_i$. Since the probabilities for all these states must sum to 1, the number of independent parameters is one less than the total number of states.

$$\text{Total independent parameters} = \left(\prod_{i=1}^{n} k_i \right) - 1$$

Alternative Derivation (Chain Rule): Using the chain rule $P(X_1, \ldots, X_n) = P(X_1)P(X_2|X_1) \cdots P(X_n|X_1$

- $P(X_1)$: requires $k_1 - 1$ parameters.
- $P(X_2|X_1)$: for each of the k_1 outcomes of X_1, we need $k_2 - 1$ parameters. Total: $k_1(k_2 - 1)$.
- $P(X_i|X_1, \ldots, X_{i-1})$: requires $(\prod_{j=1}^{i-1} k_j)(k_i - 1)$ parameters.

Summing these parameters gives a telescoping series:

$$\begin{aligned}\text{Total} &= (k_1 - 1) + k_1(k_2 - 1) + k_1k_2(k_3 - 1) + \cdots + (k_1 \cdots k_{n-1})(k_n - 1) \\ &= (k_1 - 1) + (k_1k_2 - k_1) + (k_1k_2k_3 - k_1k_2) + \cdots + (k_1 \cdots k_n - k_1 \cdots k_{n-1}) \\ &= -1 + k_1 - k_1 + k_1k_2 - k_1k_2 + \cdots - k_1 \cdots k_{n-1} + k_1 \cdots k_n \\ &= \left(\prod_{i=1}^{n} k_i\right) - 1\end{aligned}$$

Part 2: Limited Conditional Independence

Question: For every $i > m$ (where $m \in \{1, \ldots, n-1\}$), assume X_i is conditionally independent of all ancestors given its m previous ancestors: $P(X_i|X_{i-1}, \ldots, X_1) = P(X_i|X_{i-1}, \ldots, X_{i-m})$. For $i \leq m$, no conditional independence is assumed. Derive the total number of independent parameters.

Solution: We sum the parameters for each term in the chain rule factorization, applying the simplification where appropriate. The number of parameters for $P(X_i|\text{Parents})$ is (num_parent_configs) $\times$ $(k_i - 1)$.

- **Case 1:** $1 \leq i \leq m$ (No simplification)
  The parents of X_i are $(X_1, \ldots, X_{i-1})$. Number of parent configurations is $\prod_{j=1}^{i-1} k_j$. Parameters for $P(X_i|X_1, \ldots, X_{i-1}) = (\prod_{j=1}^{i-1} k_j)(k_i - 1)$. (For $i = 1$, this is $1 \times (k_1 - 1)$).
- **Case 2:** $m < i \leq n$ (Conditional independence applies)
  The effective parents of X_i are $(X_{i-m}, \ldots, X_{i-1})$. Number of parent configurations is $\prod_{j=i-m}^{i-1} k_j$. Parameters for $P(X_i|X_{i-m}, \ldots, X_{i-1}) = (\prod_{j=i-m}^{i-1} k_j)(k_i - 1)$.

Total Number of Independent Parameters: Summing the parameters from all terms:

$$\text{Total Parameters} = \underbrace{(k_1 - 1) + \sum_{i=2}^{m} \left[\left(\prod_{j=1}^{i-1} k_j\right)(k_i - 1)\right]}_{\text{for } i \leq m} + \underbrace{\sum_{i=m+1}^{n} \left[\left(\prod_{j=i-m}^{i-1} k_j\right)(k_i - 1)\right]}_{\text{for } i > m}$$

This can be written more compactly as:

$$\text{Total Parameters} = \sum_{i=1}^{m} \left[\left(\prod_{j=1}^{i-1} k_j\right)(k_i - 1)\right] + \sum_{i=m+1}^{n} \left[\left(\prod_{j=i-m}^{i-1} k_j\right)(k_i - 1)\right]$$

where the product over an empty set of indices is defined as 1 (for the $i = 1$ term).

Table 1: Parameter Calculation Breakdown by Variable (Illustrative Example for Part 2)

Variable (X_i)	**Effective Parents**	**# Parent Configs**	**Parameters for** $P(X_i|\textbf{Parents})$
X_1	$\emptyset$	1	$k_1 - 1$
X_2	X_1	k_1	$k_1(k_2 - 1)$
$\ldots$	$\ldots$	$\ldots$	$\ldots$
X_m	$X_1, \ldots, X_{m-1}$	$\prod_{j=1}^{m-1} k_j$	$(\prod_{j=1}^{m-1} k_j)(k_m - 1)$
X_{m+1}	$X_1, \ldots, X_m$	$\prod_{j=1}^{m} k_j$	$(\prod_{j=1}^{m} k_j)(k_{m+1} - 1)$
X_{m+2}	$X_2, \ldots, X_{m+1}$	$\prod_{j=2}^{m+1} k_j$	$(\prod_{j=2}^{m+1} k_j)(k_{m+2} - 1)$
$\ldots$	$\ldots$	$\ldots$	$\ldots$
X_n	$X_{n-m}, \ldots, X_{n-1}$	$\prod_{j=n-m}^{n-1} k_j$	$(\prod_{j=n-m}^{n-1} k_j)(k_n - 1)$

Part 3: Full Independence

Question: Under what independence assumptions can the joint distribution be represented with $\sum_{i=1}^{n}(k_i - 1)$ total independent parameters?

Solution: The sum $\sum_{i=1}^{n}(k_i - 1)$ represents the number of parameters if each variable X_i is specified independently of all other variables. This corresponds to a joint distribution that factorizes as:

$$P(X_1, \ldots, X_n) = P(X_1)P(X_2)\cdots P(X_n) = \prod_{i=1}^{n} P(X_i)$$

The number of parameters for each term $P(X_i)$ is $k_i - 1$. Summing these gives the total. This scenario corresponds to the assumption of **full mutual independence** among all variables $X_1, \ldots, X_n$.

In terms of the conditional independence assumption from Part 2, this situation arises if we set $m = 0$. If $m = 0$, then for any $i > 0$, the assumption $P(X_i|X_{i-1}, \ldots, X_1) = P(X_i|X_{i-1}, \ldots, X_{i-0})$ means $P(X_i|X_{i-1}, \ldots, X_1) = P(X_i|\emptyset) = P(X_i)$. This implies each X_i is independent of all its predecessors, which for a topological sort implies full mutual independence.

4 Problem 4: Autoregressive Models

This problem investigates the hypothesis spaces of forward and reverse Gaussian autoregressive models, questioning whether they can represent the same set of probability distributions.

4.1 A. Conceptual Deep Dive: Sequential Data Modeling

4.1.1 Autoregressive (AR) Models: Fundamentals

Autoregressive (AR) models capture dependencies in sequential data by predicting the current value of a variable based on its own preceding values.[21] The joint probability distribution $P(x_1, \ldots, x_n)$ is factorized using the chain rule.

- **Forward model:** $p_f(x_1, \ldots, x_n) = \prod_{i=1}^{n} p_f(x_i|x_{<i})$
- **Reverse model:** $p_r(x_1, \ldots, x_n) = \prod_{i=n}^{1} p_r(x_i|x_{>i})$

In a **Gaussian autoregressive model**, each conditional distribution is assumed to be Gaussian.

- Forward: $p_f(x_i|x_{<i}) = \mathcal{N}(x_i|\mu_i(x_{<i}), \sigma_i^2(x_{<i}))$
- Reverse: $p_r(x_i|x_{>i}) = \mathcal{N}(x_i|\hat{\mu}_i(x_{>i}), \hat{\sigma}_i^2(x_{>i}))$

The **hypothesis space** of a model class is the set of all probability distributions that can be represented by that class. The question is whether the forward and reverse models have the same hypothesis space.

4.2 B. Solution and Proof/Counterexample for Problem 4

Objective: Determine if the forward Gaussian autoregressive model p_f and the reverse order model p_r cover the same hypothesis space of distributions.

The answer depends critically on the allowed functional forms for the conditional means and variances.

4.2.1 Case 1: Linear Gaussian AR Models (Multivariate Gaussian)

If we interpret "Gaussian autoregressive model" in the classic sense where the joint distribution $P(X_1, \ldots, X_n)$ is multivariate Gaussian, then the conditional means $\mu_i(\cdot)$ must be linear functions of their context, and the conditional variances $\sigma_i^2(\cdot)$ must be constant.

Conclusion: Yes, they cover the same hypothesis space.

Proof for n=2: A general bivariate Gaussian distribution for (X_1, X_2) is defined by 5 parameters: means $(\mathbb{M}_1, \mathbb{M}_2)$, variances $(\mathbb{S}_1^2, \mathbb{S}_2^2)$, and correlation ρ.

- **Forward Model** $p_f(x_1, x_2) = p_f(x_1)p_f(x_2|x_1)$**:** $p_f(x_1) = \mathcal{N}(x_1|\mu_1, \sigma_1^2)$ and $p_f(x_2|x_1) = \mathcal{N}(x_2|ax_1 + b, \sigma_{2|1}^2)$. These parameters uniquely define a bivariate Gaussian.
- **Reverse Model** $p_r(x_1, x_2) = p_r(x_2)p_r(x_1|x_2)$**:** $p_r(x_2) = \mathcal{N}(x_2|\hat{\mu}_2, \hat{\sigma}_2^2)$ and $p_r(x_1|x_2) = \mathcal{N}(x_1|cx_2 + d, \hat{\sigma}_{1|2}^2)$. These also uniquely define a bivariate Gaussian.

Any bivariate Gaussian distribution can be factorized in either the forward or reverse direction. The parameters of one factorization can be uniquely determined from the parameters of the other by first computing the 5 parameters of the joint distribution and then deriving the conditional

parameters for the other direction. For example, from the forward parameters, we can find the joint parameters, and from the joint parameters, we can find the reverse parameters:

$$\begin{aligned}\hat{\mu}_2 &= \mathbb{M}_2 = a\mu_1 + b \\ \hat{\sigma}_2^2 &= \mathbb{S}_2^2 = a^2\sigma_1^2 + \sigma_{2|1}^2 \\ c &= \rho\frac{\mathbb{S}_1}{\mathbb{S}_2} \quad \text{and} \quad d = \mathbb{M}_1 - c\mathbb{M}_2 \\ \hat{\sigma}_{1|2}^2 &= \mathbb{S}_1^2(1-\rho^2)\end{aligned}$$

Since this transformation is always possible for any non-degenerate bivariate Gaussian, the two model classes cover the same hypothesis space: the set of all bivariate Gaussian distributions. This extends to the general n-variate case.

4.2.2 Case 2: General Non-linear AR Models (as per problem notation)

The problem notation allows $\mu_i(\cdot)$ and $\sigma_i^2(\cdot)$ to be arbitrary measurable functions. This allows for highly non-Gaussian joint distributions.

Conclusion: No, they do not cover the same hypothesis space.

Counterexample for n=2: Let the forward model p_f be defined as:

- $p_f(x_1) = \mathcal{N}(x_1|0,1)$
- $p_f(x_2|x_1) = \mathcal{N}(x_2|x_1^3,1)$ (Here, $\mu_2(x_1) = x_1^3$ is non-linear).

This defines a valid joint distribution $p_f(x_1,x_2) = \mathcal{N}(x_1|0,1)\mathcal{N}(x_2|x_1^3,1)$. This joint distribution is non-Gaussian.

Now, consider the reverse model $p_r(x_1,x_2) = p_r(x_2)p_r(x_1|x_2)$. For p_r to represent p_f, its marginal distribution $p_r(x_2)$ must be equal to the marginal of $p_f(x_1,x_2)$ with respect to x_1:

$$p_r(x_2) = \int_{-\infty}^{\infty} p_f(x_1,x_2)dx_1 = \int_{-\infty}^{\infty} \frac{1}{\sqrt{2\pi}}\exp\left(-\frac{x_1^2}{2}\right)\frac{1}{\sqrt{2\pi}}\exp\left(-\frac{(x_2-x_1^3)^2}{2}\right)dx_1$$

This integral for $p_r(x_2)$ is generally not a simple Gaussian distribution. It will be a more complex, potentially multi-modal distribution.

However, the definition of the reverse Gaussian AR model p_r requires its "starting" distribution $p_r(x_2)$ to be of the form $\mathcal{N}(x_2|\hat{\mu}_2,\hat{\sigma}_2^2)$, where $\hat{\mu}_2$ and $\hat{\sigma}_2^2$ are constants (since the context $x_{>2}$ is empty).

Since the marginal $p_f(x_2)$ derived from the non-linear forward model is not a Gaussian, the reverse model p_r (which is constrained to have a Gaussian marginal $p_r(x_2)$) cannot represent this specific distribution p_f. Therefore, the hypothesis spaces are not the same.

5 Problem 5: Monte Carlo Integration

This problem addresses the estimation of intractable integrals that arise in latent variable models, specifically the marginal likelihood, using Monte Carlo methods.

5.1 A. Conceptual Deep Dive: Estimating Intractable Integrals

5.1.1 Latent Variable Models and Marginal Likelihood

Latent variable models posit that observed data x are generated via unobserved (latent) variables z. The model specifies a joint distribution $p(x, z) = p(z)p(x|z)$. A key quantity is the **marginal likelihood** (or model evidence), $p(x)$, obtained by integrating out the latent variables:

$$p(x) = \int p(x, z)dz = \int p(z)p(x|z)dz = \mathbb{E}_{z\sim p(z)}[p(x|z)]$$

This integral is often analytically intractable.

5.1.2 Monte Carlo Estimation

Monte Carlo methods approximate an integral $\mathbb{E}_{p(z)}[f(z)]$ by drawing k i.i.d. samples $z^{(1)}, \ldots, z^{(k)}$ from $p(z)$ and computing the sample mean:[31]

$$\hat{I}_k = \frac{1}{k}\sum_{i=1}^{k} f(z^{(i)})$$

For the marginal likelihood, we set $f(z) = p(x|z)$, giving the estimator $A(z^{(1)}, \ldots, z^{(k)}) = \frac{1}{k}\sum_{i=1}^{k} p(x|z^{(i)})$.

5.1.3 Unbiased Estimators and Jensen's Inequality

An estimator $\hat{\theta}$ is **unbiased** if $\mathbb{E}[\hat{\theta}] = \theta$.[31] **Jensen's inequality** states that for a convex function g, $\mathbb{E}[g(X)] \geq g(\mathbb{E}[X])$. For a strictly concave function like $g(x) = \log(x)$, the inequality is reversed and strict for non-constant random variables: $\mathbb{E}[\log(X)] < \log(\mathbb{E}[X])$.[35]

5.2 B. Solution and Explanation for Problem 5

Part 1: Show that A is an unbiased estimator of $p(x)$

The estimator is $A(z^{(1)}, \ldots, z^{(k)}) = \frac{1}{k}\sum_{i=1}^{k} p(x|z^{(i)})$, where $z^{(i)} \sim p(z)$ are i.i.d. samples. We want to compute the expectation of A.

$$\begin{aligned}
\mathbb{E}[A] &= \mathbb{E}\left[\frac{1}{k}\sum_{i=1}^{k} p(x|z^{(i)})\right] \\
&= \frac{1}{k}\mathbb{E}\left[\sum_{i=1}^{k} p(x|z^{(i)})\right] && \text{(Linearity of Expectation)} \\
&= \frac{1}{k}\sum_{i=1}^{k} \mathbb{E}[p(x|z^{(i)})] && \text{(Linearity of Expectation)}
\end{aligned}$$

Now, consider the expectation for a single term $\mathbb{E}[p(x|z^{(i)})]$. Since each $z^{(i)}$ is drawn from $p(z)$, this expectation is taken with respect to $p(z)$:

$$\mathbb{E}[p(x|z^{(i)})] = \int p(x|z)p(z)dz$$

This integral is the definition of the marginal likelihood $p(x)$. So, for each i, $\mathbb{E}[p(x|z^{(i)})] = p(x)$. Substituting this back into the expression for $\mathbb{E}[A]$:

$$\mathbb{E}[A] = \frac{1}{k}\sum_{i=1}^{k} p(x) = \frac{1}{k}(k \cdot p(x)) = p(x)$$

Thus, A is an unbiased estimator of $p(x)$.

Part 2: Is $\log A$ an unbiased estimator of $\log p(x)$?

We are asked to determine if $\mathbb{E}[\log A] = \log p(x)$.

The function $g(u) = \log u$ is a strictly concave function for $u > 0$. The estimator $A = \frac{1}{k}\sum_{i=1}^{k} p(x|z^{(i)})$ is a random variable, as its value depends on the random samples $z^{(1)}, \ldots, z^{(k)}$. For any finite k and non-trivial model where $p(x|z)$ is not constant, A is a non-constant random variable.

By **Jensen's inequality** for a strictly concave function and a non-constant positive random variable A:[35]

$$\mathbb{E}[\log A] < \log(\mathbb{E}[A])$$

From Part 1, we know that $\mathbb{E}[A] = p(x)$. Substituting this into the inequality:

$$\mathbb{E}[\log A] < \log(p(x))$$

Since $\mathbb{E}[\log A] \neq \log p(x)$, the estimator $\log A$ is **biased**. Specifically, it is a **negatively biased** estimator, meaning it tends to underestimate $\log p(x)$ on average. The equality would only hold if A were a constant, which is not the case for a Monte Carlo estimator with finite samples and non-zero variance.

6 Problem 6: Programming Assignment (LSTM for Text Generation)

This problem involves working with a pretrained autoregressive generative model, specifically a four-layer LSTM, for text generation and analysis tasks.

6.1 A. Conceptual Deep Dive: LSTMs for Autoregressive Text Modeling

6.1.1 Representing Characters: Binary Encoding

To represent a set of M unique items using a binary code of length n, we need enough unique binary patterns to assign one to each item. With n bits, we can form 2^n unique patterns. Therefore, we must have $2^n \geq M$. To find the minimal integer value of n, we take the base-2 logarithm: $n \geq \log_2(M)$. Since n must be an integer, the minimal value is $n = \lceil \log_2(M) \rceil$.

6.1.2 LSTM Architecture and Parameters

The provided model architecture (Figure 1 in the problem description, not shown here) consists of several components with trainable parameters:

1. **Embedding Layer:** Maps each character index (from a vocabulary of size V) to a dense vector of dimension E. Parameters: $V \times E$.
2. **LSTM Layers:** Process the sequence of embeddings. The number of parameters depends on the input dimension and the hidden state dimension H.
3. **Fully-Connected Layer:** Takes the output from the final LSTM layer (dimension H) and transforms it into a vector of logits of dimension V. Parameters: $(H \times V) + V$.

6.1.3 Autoregressive Text Generation and Sampling

Autoregressive models generate sequences one token at a time, where each new token is conditioned on all previously generated tokens. The joint probability of a sequence $C = (c_1, \ldots, c_L)$ is factorized as:

$$P(C) = \prod_{i=1}^{L} P(c_i | c_{<i})$$

Ancestral sampling is the process of generating a sequence from such a model by iteratively sampling the next token from the model's predicted probability distribution and feeding it back as input for the next step.

6.1.4 Log-Likelihood for Text Sequences and Classification

The log-likelihood of a given text sequence $C = (c_1, \ldots, c_L)$ under an autoregressive model is the sum of the log-probabilities of generating each token given its predecessors:

$$\log P(C) = \sum_{i=1}^{L} \log P(c_i | c_1, \ldots, c_{i-1})$$

This value can be used for classification. A model trained on a specific domain (e.g., NIPS papers) will assign a higher log-likelihood to new text from that same domain compared to text from a different domain (e.g., Shakespeare).

6.2 B. Solutions and Explanations for Programming Tasks

Part 1: Minimal bits for character representation

Question: To represent 657 unique characters with a binary code, what is the minimal value of n?

Solution: We need to find the smallest integer n such that $2^n \geq 657$.

$$n \geq \log_2(657) \approx 9.359$$

Since n must be an integer, we take the ceiling:

$$n = \lceil 9.359 \rceil = 10$$

The minimal value of n is 10 bits.

Part 2: Increase in model parameters

Question: If the vocabulary size increases from 657 to 900, what is the increase in the number of model parameters? (Consider only embedding and fully-connected layers).

Solution: Let $V_{\text{old}} = 657$, $V_{\text{new}} = 900$, E be the embedding dimension, and H be the LSTM hidden dimension.

- **Increase in Embedding Layer Parameters:** $\Delta P_{\text{emb}} = (V_{\text{new}} \times E) - (V_{\text{old}} \times E) = (900 - 657)E = 243E$.
- **Increase in Fully-Connected Layer Parameters:** $\Delta P_{\text{fc}} = (V_{\text{new}}(H+1)) - (V_{\text{old}}(H+1)) = (900 - 657)(H+1) = 243(H+1)$.

Total Increase in Parameters:

$$\Delta P_{\text{total}} = \Delta P_{\text{emb}} + \Delta P_{\text{fc}} = 243E + 243(H+1) = \mathbf{243(E + H + 1)}$$

Table 2: Parameter Calculation for Vocabulary Change

Layer	**Parameters (V=657)**	**Parameters (V=900)**	**Increase**
Embedding Layer	$657E$	$900E$	$243E$
Fully-Connected Output	$657(H+1)$	$900(H+1)$	$243(H+1)$
Total Increase			$\mathbf{243(E + H + 1)}$

Part 3: Complete the ‘sample‘ method

The ‘sample‘ method should implement ancestral sampling.

1. Initialize hidden state and process any seed characters to prime the model.
2. Loop for the desired number of characters to generate.
3. In each step, feed the last generated character and the current hidden state to the model.
4. Obtain the output logits and the new hidden state.
5. Apply the temperature parameter to the logits: ‘scaled$_l ogits = logits/temperature$‘.$Convertscaledlogitst$

6. Sample the next character’s index from this distribution.
7. Convert the index back to a character, append it to the output, and use it as the input for the next step.
8. Repeat 5 times to generate 5 paragraphs of 1000 characters each.

Part 4: Complete the 'compute$_p$*rob*'*methodandplothistograms*

The 'compute$_p$*rob*'*methodcalculatesthelog* − *likelihoodofastring*.

Initialize hidden state and a total log-likelihood of 0.

Iterate through the input string, from the first character to the second-to-last.

In each step 't', use character 'c$_t$'*asinputtopredictthedistributionfor*'c_{t+1}'.*Gettheprobabilitythemodelass*

Add the log of this probability to the total.

Return the total log-likelihood.

For plotting, this method should be called on all strings from 'nips.txt' and 'shakespeare.txt'. The resulting lists of log-likelihoods are then used to generate two separate histograms, showing the distribution of scores for each category under the NIPS-trained model.

Part 5: Infer categories of new strings

This task uses the log-likelihood as a classification score.

1. **Establish Baselines:** The histograms from Part 4 establish the typical log-likelihood ranges for "NIPS" and "Shakespeare" texts. A third baseline for "Random" text can be generated by computing log-likelihoods for random character strings. We expect:

$$\text{LL(NIPS)} > \text{LL(Shakespeare)} > \text{LL(Random)}$$

2. **Infer Categories:** For each new string in 'snippets.pkl':

   (a) Compute its log-likelihood using the 'compute$_p$*rob*'*method.Comparethisvaluetothebaselinedis*

(b) Classify the snippet into the category whose log-likelihood range best matches the snippet's score. For example, a very high score suggests "NIPS", a moderately low score suggests "Shakespeare", and a very low score suggests "Random".

Table 3: Example Inferred Categories for 'snippets.pkl'

Snippet ID/Text (truncated)	Log-Likelihood	Inferred Category	Justification
"The method proposed..."	-150.5	NIPS publication	High LL, consistent with NIPS
"To be, or not to be..."	-450.2	Shakespeare	Moderate LL, lower than NIPS
"axq Zj yyp qqq..."	-950.8	Randomly generated	Very low LL, inconsistent with

Conclusion

This report has traversed a series of advanced topics in probabilistic machine learning, providing both theoretical elucidations and solutions to specific problems. Key concepts explored include the equivalence of MLE and KL-divergence minimization, the relationship between generative and discriminative models, the power of conditional independence for parameter reduction, the properties of autoregressive models, the use of Monte Carlo methods for intractable integrals, and the application of LSTMs for text generation and classification. A thorough grasp of these theoretical underpinnings, combined with the ability to implement and analyze models, forms a strong foundation for further work and research in machine learning.